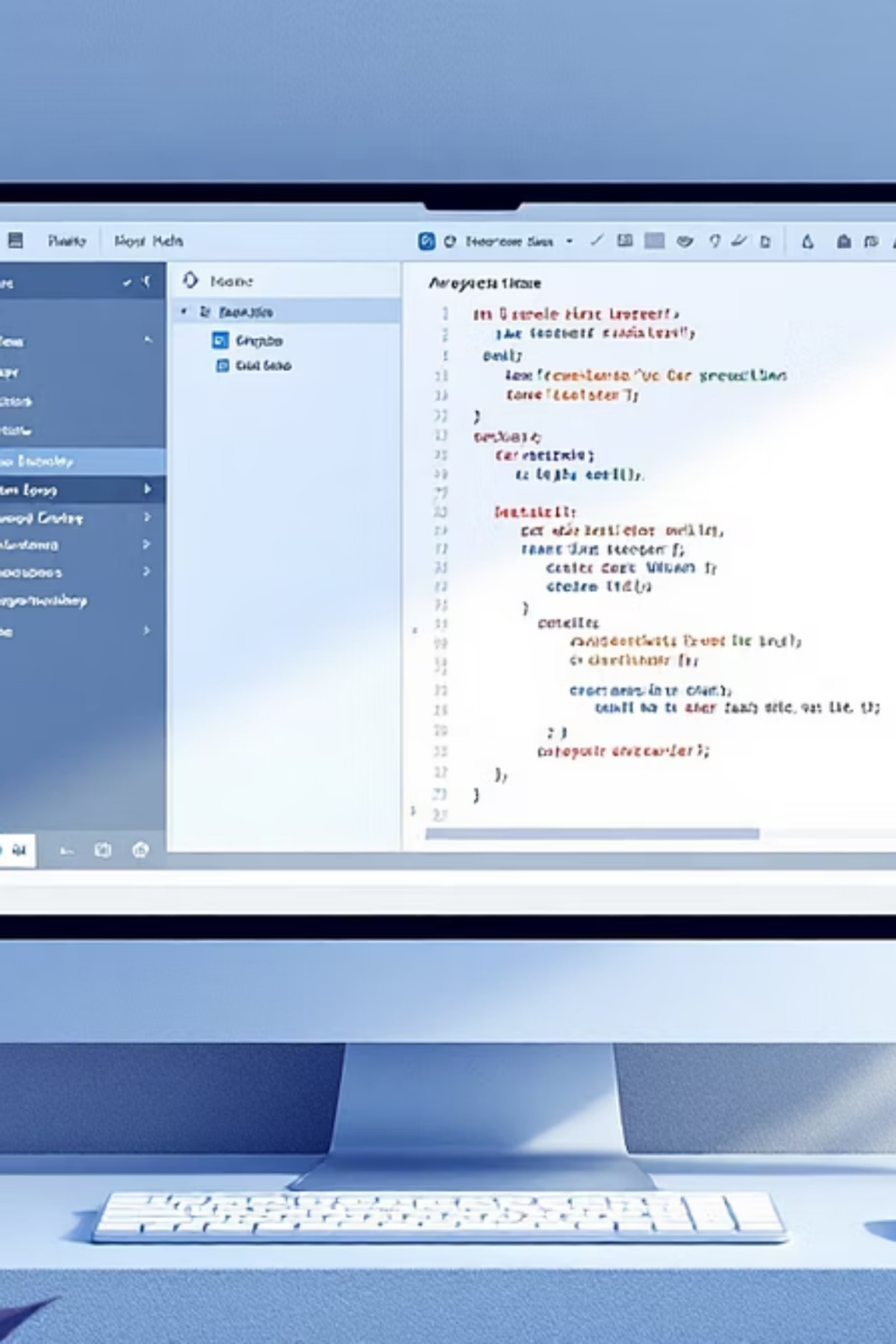




XSLT y XSL: Transformación de XML

Los ficheros XML permiten organizar la información de forma jerárquica y estructurada, pero en su estado bruto son difíciles de consumir directamente por usuarios finales o sistemas externos. Para extraer su verdadero valor, necesitamos herramientas capaces de transformar, formatear y presentar esa información de manera significativa.

XSLT es el estándar que hace posible todo esto, convirtiendo documentos XML en HTML, PDF, otros XML y mucho más.

¿Por qué necesitamos transformar XML?

Los documentos XML son legibles por máquinas y están bien estructurados, pero presentarlos directamente a un usuario final o integrarlo en otro sistema sin procesamiento previo limita enormemente su utilidad.

La transformación XML es el puente entre los datos en bruto y la información con valor añadido.

El problema

Un fichero XML contiene datos estructurados jerárquicamente, pero su visualización directa en un navegador o aplicación resulta técnica e incomprensible para la mayoría de usuarios. Se necesita una capa de transformación que convierta esos datos en algo legible, imprimible o procesable por otro sistema.

- Información jerárquica difícil de leer.
- Necesidad de múltiples formatos de salida.
- Integración con sistemas heterogéneos.
- Presentación orientada al usuario final.

La solución: XSL

XSL (Extensible Stylesheet Language) es mucho más que un simple lenguaje de hojas de estilo. A diferencia de CSS, que únicamente cambia la apariencia visual de un documento sin modificar su estructura ni contenido, **XSL transforma completamente un documento en otro**. Es decir, el documento resultante puede tener una estructura, formato y contenido completamente diferente al original.

- Transformación real del documento.
- Generación de múltiples formatos de salida.
- Reestructuración completa del contenido.
- Filtrado, ordenación y lógica condicional.

La familia XSL: tres tecnologías complementarias

XSL no es una única tecnología, sino un conjunto de tres estándares del W3C que trabajan de forma complementaria para cubrir distintas necesidades en el procesamiento de documentos XML.

Cada uno tiene un propósito específico y bien definido.

XPath

Navegación entre nodos

XPath es el lenguaje que permite navegar por la estructura jerárquica de un documento XML para seleccionar nodos o conjuntos de nodos específicos. Es la base sobre la que se construyen las expresiones de selección en XSLT. Permite acceder a elementos, atributos, textos y nodos de comentario mediante rutas de localización.

- Selección de nodos por ruta.
- Filtros y predicados.
- Funciones de cadena, número y booleano.

XSLT

Transformación de documentos

XSLT (Extensible Stylesheet Language Transformations) es el lenguaje principal para transformar un documento XML de origen en otro documento de destino, que puede ser XML, HTML, texto plano u otros formatos. Utiliza plantillas (*templates*) que se emparejan con nodos XML mediante expresiones XPath. Actualmente en su versión 3.0.

- Conversión XML → HTML, XML, texto.
- Plantillas basadas en patrones.
- Lógica condicional e iteración.

XSL-FO

Formateo de objetos

XSL-FO (XSL Formatting Objects) es el componente de XSL dedicado al formateo visual de documentos XML para su presentación o impresión. Permite definir con precisión la maquetación de páginas, márgenes, tipografías y estilos visuales avanzados, siendo especialmente útil para generar documentos PDF de alta calidad a partir de datos XML.

- Maquetación de páginas para impresión.
- Generación de documentos PDF.
- Control tipográfico avanzado.

XSLT 3.0: La evolución del estándar

El hecho de que XSLT haya alcanzado su versión 3.0 es una señal inequívoca de la madurez e importancia de este estándar en el ecosistema tecnológico. Cada versión ha incorporado nuevas capacidades que amplían significativamente su potencia y versatilidad.

XSLT 1.0 (1999)

Primera versión oficial del W3C. Estableció las bases del lenguaje con plantillas, iteración y condicionales. Ampliamente soportada por todos los procesadores y navegadores.

1

2

3

XSLT 3.0 (2017)

Versión más reciente y potente. Introduce **streaming** de documentos fuente (procesamiento sin cargar el documento completo en memoria), mapas y arrays, funciones de orden superior y mejor integración con JSON.

XSLT 2.0 (2007)

Incorporó tipos de datos de XML Schema, funciones de agrupación (*grouping*), expresiones regulares, múltiples documentos de salida y una mayor integración con XPath 2.0.

📌 **Novedad destacada en XSLT 3.0:** El *streaming* permite procesar documentos XML de gran tamaño sin necesidad de cargarlos completamente en memoria, lo que supone una ventaja crítica en entornos con grandes volúmenes de datos o recursos limitados.

Vinculando XML con su hoja de estilos XSL

Para que un procesador XSL sepa qué hoja de estilos debe aplicar a un documento XML, es necesario incluir una instrucción de procesamiento en el propio fichero XML. Esta instrucción establece el vínculo entre el documento de datos y la hoja de transformación.

Instrucción de procesamiento en el XML

Se añade en el preámbulo del documento XML, justo después de la declaración XML estándar. Indica al procesador el tipo de hoja de estilos (`text/xsl`) y la ruta al fichero XSL mediante el atributo `href`:

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="RAQUETAS.XSL"?>
<RAQUETAS>
  <RAQUETA>
    <MARCA>Wilson</MARCA>
    <MODELO>Pro Staff</MODELO>
    <PRECIO>299.99</PRECIO>
  </RAQUETA>
</RAQUETAS>
```

Estructura básica de un fichero XSL

La hoja de estilos XSL es en sí misma un documento XML bien formado. Declara el espacio de nombres de XSLT y define las plantillas que serán aplicadas sobre los nodos del documento fuente:

```
<?xml version="1.0"?>
<xsl:stylesheet
  version="1.0"
  xmlns:xsl=
    "http://www.w3.org/1999/XSL/Transform">

  <xsl:template match="/">
    <html>
      <body>
        ...
      </body>
    </html>
  </xsl:template>

</xsl:stylesheet>
```

El atributo `match="/"` en la instrucción `xsl:template` utiliza XPath para asociar la plantilla al nodo raíz del documento XML. Desde ahí, la transformación puede recorrer toda la jerarquía del documento para generar la salida deseada.

Instrucciones XSLT esenciales

XSLT dispone de un conjunto de instrucciones bien definidas que permiten construir transformaciones complejas. Dominar estas etiquetas fundamentales es el primer paso para trabajar eficazmente con hojas de estilos XSL. A continuación se describen las más utilizadas en el día a día:

xsl:template

Es la instrucción central de XSLT. Define una plantilla que se activa cuando el procesador encuentra un nodo que coincide con el patrón XPath especificado en el atributo `match`. Por ejemplo, `match="RAQUETA"` activará la plantilla para cada elemento `<RAQUETA>` del XML fuente. Es la unidad básica de toda transformación XSLT.

xsl:value-of

Extrae el valor de texto de un elemento XML o atributo seleccionado mediante una expresión XPath en el atributo `select`, y lo añade directamente al flujo de salida de la transformación. Por ejemplo, `<xsl:value-of select="MARCA"/>` inserta el texto contenido en el elemento `<MARCA>` en el documento resultante. Muy utilizado para volcar datos concretos en la salida.

xsl:for-each

Permite iterar sobre un conjunto de nodos seleccionados mediante una expresión XPath en el atributo `select`, procesando cada nodo individualmente. Es el equivalente a un bucle *for* en lenguajes de programación convencionales. Se utiliza, por ejemplo, para generar una fila de tabla HTML por cada elemento `<RAQUETA>` del documento XML. Puede anidarse para estructuras complejas.

xsl:sort

Se utiliza dentro de `xsl:for-each` o `xsl:apply-templates` para ordenar los nodos antes de procesarlos. El atributo `select` indica el criterio de ordenación (un campo del XML) y el atributo `order` permite elegir entre `ascending` o `descending`. También admite el atributo `data-type` para distinguir entre ordenación textual y numérica.

xsl:if

Estructura condicional básica de XSLT. El contenido de la instrucción se incluye en la salida únicamente si la expresión XPath del atributo `test` se evalúa como verdadera. A diferencia de los lenguajes de programación convencionales, `xsl:if` no dispone de una cláusula `else`; para ello se debe utilizar `xsl:choose`. Ejemplo de uso: mostrar una celda destacada solo si el precio supera un umbral.

xsl:choose

Estructura condicional múltiple equivalente al `switch/case` de otros lenguajes. Contiene una o más cláusulas `xsl:when` (cada una con su propio atributo `test`) y, opcionalmente, una cláusula `xsl:otherwise` que actúa como caso por defecto cuando ninguna condición se cumple. Ideal para aplicar diferentes formatos o contenidos según el valor de un campo XML.

Instrucciones en acción: ejemplos prácticos

Ver las instrucciones XSLT aplicadas en código real facilita enormemente su comprensión. Los siguientes fragmentos ilustran cómo combinar las instrucciones más habituales para construir transformaciones útiles y completas.

Iteración con `xsl:for-each` y ordenación con `xsl:sort`

```
<xsl:template match="/">
<html><body>
  <table border="1">
    <tr>
      <th>Marca</th>
      <th>Precio</th>
    </tr>
    <xsl:for-each select="RAQUETAS/RAQUETA">
      <xsl:sort select="PRECIO"
        data-type="number"
        order="ascending"/>
      <tr>
        <td>
          <xsl:value-of select="MARCA"/>
        </td>
        <td>
          <xsl:value-of select="PRECIO"/>
        </td>
      </tr>
    </xsl:for-each>
  </table>
</body></html>
</xsl:template>
```

Condicional múltiple con `xsl:choose`

```
<xsl:for-each select="RAQUETAS/RAQUETA">
  <tr>
    <td>
      <xsl:value-of select="MARCA"/>
    </td>
    <td>
      <xsl:choose>
        <xsl:when test="PRECIO > 300">
          <span style="color:red">
            Premium
          </span>
        </xsl:when>
        <xsl:when test="PRECIO > 150">
          <span style="color:orange">
            Estándar
          </span>
        </xsl:when>
        <xsl:otherwise>
          Económica
        </xsl:otherwise>
      </xsl:choose>
    </td>
  </tr>
</xsl:for-each>
```

XSL vs. CSS: una diferencia fundamental

Una confusión frecuente entre quienes se acercan a XSL por primera vez es equiparlo con CSS, dado que ambos se presentan como "hojas de estilos". Sin embargo, su naturaleza y alcance son radicalmente distintos.

CSS — Presentación Visual

CSS (Cascading Style Sheets) actúa sobre un documento HTML o XML **sin modificar su estructura ni contenido**. Simplemente altera la apariencia visual: colores, tipografías, márgenes, animaciones. El documento sigue siendo exactamente el mismo; solo cambia cómo se muestra al usuario.

- Opera sobre el documento original
- No cambia la estructura del DOM
- Solo afecta a la presentación visual
- No puede reordenar, filtrar ni calcular
- Ideal para estilizar HTML estático

XSL — Transformación Real

XSL va mucho más allá de la presentación. Toma un documento XML de entrada y genera un documento de salida completamente diferente. Puede cambiar su estructura, reordenar elementos, filtrar datos, calcular valores, añadir contenido nuevo y producir formatos tan distintos como HTML, PDF o un nuevo XML.

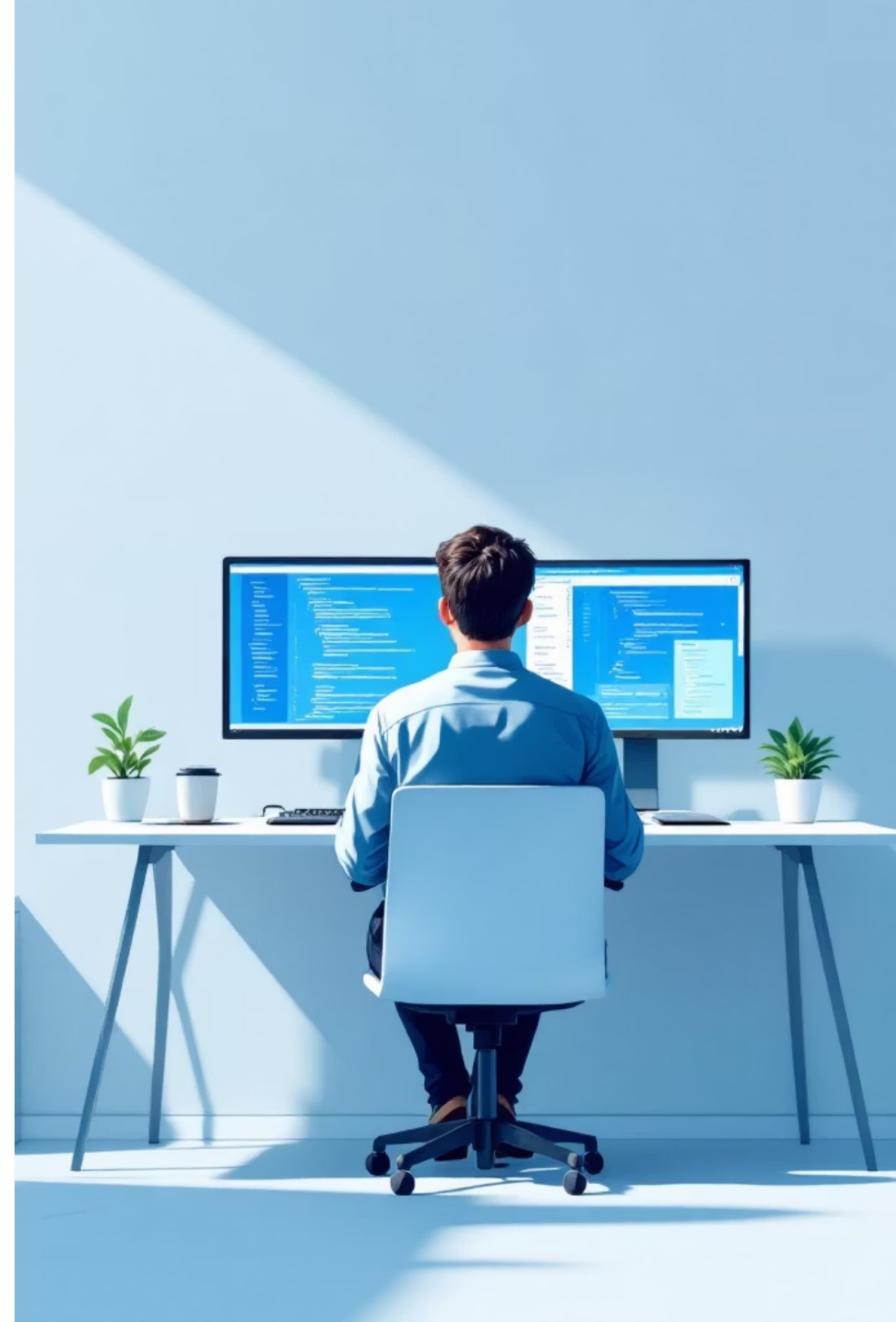
- Genera un documento nuevo y distinto
- Puede reestructurar completamente el contenido
- Permite filtrar, ordenar y calcular
- Soporta lógica condicional compleja
- Produce múltiples formatos de salida

📌 **Regla práctica:** Si necesitas que tus datos XML sean visualmente más atractivos manteniendo su estructura, usa CSS. Si necesitas convertir esos datos en otro formato, reorganizarlos o extraer información específica para un nuevo documento, necesitas XSLT.

Herramientas de procesamiento XSLT

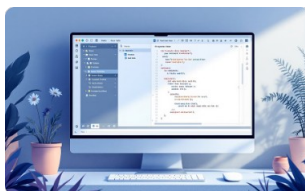
Existe un amplio ecosistema de herramientas para trabajar con XSLT, desde potentes IDEs comerciales hasta procesadores de código abierto y utilidades online.

La elección depende del contexto de uso, el presupuesto disponible y las necesidades del proyecto.



Catálogo de herramientas XSLT

A continuación se presenta una comparativa de las principales herramientas disponibles en el mercado, organizadas por categoría, para ayudarte a elegir la más adecuada según tus necesidades y contexto de trabajo.



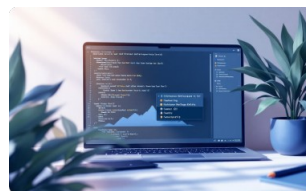
Herramientas de pago

Orientadas a entornos profesionales y empresariales. Incluyen depurador paso a paso, validación avanzada, autocompletado inteligente y soporte para las últimas versiones de XSLT, XPath y XQuery. Son la opción preferida para proyectos complejos o de gran escala.

oXygen XML Editor — El estándar de facto para desarrollo XML/XSLT profesional. Depurador integrado, perfilador de rendimiento y soporte completo para XSLT 3.0.

Altova XMLSpy — Potente IDE con diseñador gráfico de hojas XSLT, depurador y generación automática de código.

Editix XML Editor — Alternativa más asequible con buenas capacidades de edición y depuración XSLT.



Herramientas open source

Procesadores robustos y de alta calidad disponibles de forma gratuita. Ideales para integración en pipelines de compilación, servidores y proyectos donde el coste de licencia es un factor determinante. Su principal limitación es un entorno de desarrollo (IDE) menos elaborado que las opciones comerciales.

Saxon — El procesador XSLT open source más completo y actualizado. Soporta XSLT 3.0, XPath 3.1 y XQuery 3.1. Disponible en versiones Java y .NET.

Xalan — Procesador XSLT 1.0 de Apache, maduro y ampliamente utilizado en entornos Java y C++.

Xpontos — Editor XML ligero con soporte para edición y transformación XSLT.



Herramientas online

Accesibles desde cualquier dispositivo y sistema operativo sin necesidad de instalación. Perfectas para pruebas rápidas, aprendizaje o verificación de transformaciones simples. Su principal limitación es la funcionalidad reducida y las restricciones en el tamaño de los documentos procesados.

XSL Test Online — Permite pegar el XML y el XSL directamente en el navegador y ver el resultado de la transformación al instante.

FreeFormatter XSL — Herramienta online con interfaz sencilla para ejecutar transformaciones XSLT 1.0 sin configuración adicional.



Recomendación: Para comenzar a aprender XSLT, las herramientas online son perfectas por su inmediatez. Para desarrollo profesional, Saxon (open source) combinado con un editor como VS Code es una opción muy potente y gratuita. Para proyectos empresariales complejos, oXygen XML Editor es la referencia indiscutible.